VITORIA-GASTEIZKO INGENIARITZA ESKOLA
ESCUELA DE INGENIERÍA DE VITORIA-GASTEIZ

GRADO EN Ingeniería Informática de Gestión y Sistemas de Información

TRABAJO FIN DE GRADO

DATA AUGMENTATION PARA EL BALANCEO Y CLASIFICACIÓN DE MOBILIARIO URBANO CON POINTNET

Curso: 4º	Fecha: 19/06/2026

Alumno/Alumna: Yerai García García

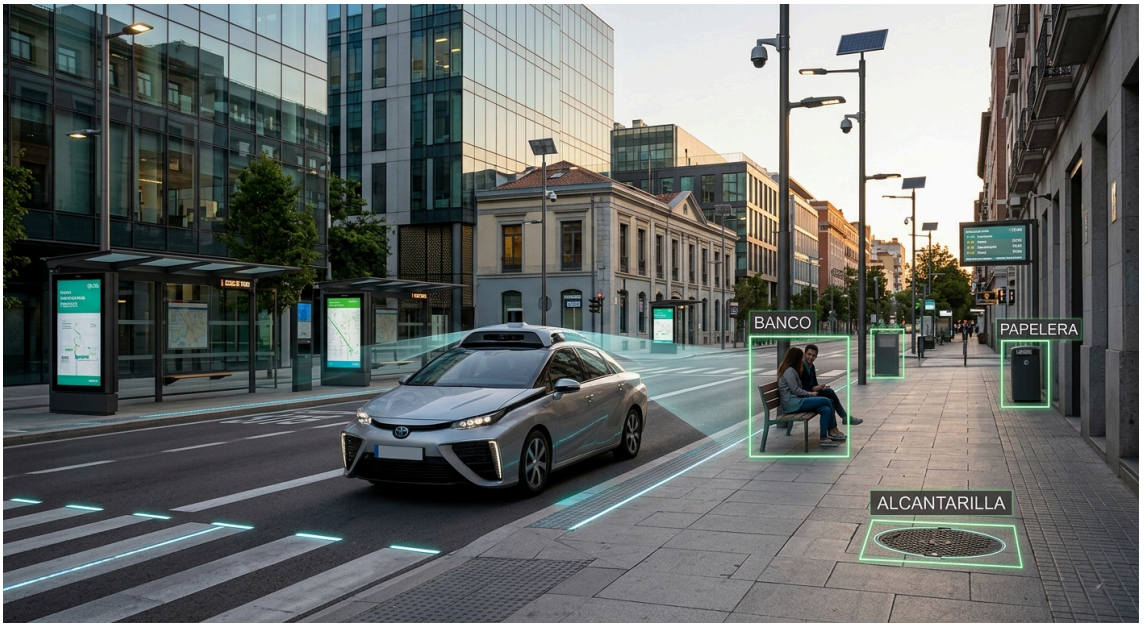
Director/Directora: José Manuel López Guede

1. Introducción 5
1.1. Contexto 6
1.2. Planteamiento del Problema 7
1.3. Objetivos 8
2. Presupuesto y Viabilidad Económica del Proyecto 8
2.1. Costes de Personal 9
2.2. Costes de Hardware (Amortización) 9
2.3. Costes de Software 9
2.4. Costes Indirectos 10
2.5. Presupuesto Total 10
3. Marco Teórico y Estado del Arte 10
3.1. Procesamiento de Nubes de Puntos 3D 11
3.2. Modelo PointNet 12
3.3. Data Augmentation en Nubes de Puntos LiDAR 13
3.4. Técnicas de Data Augmentation 14
4. Creación del Conjunto de Datos 16
4.1. Captura de datos 17
4.2. Segmentación y etiquetado 17
4.3. Análisis del DataSet 18
4.4. Partición del Dataset 19
5. Diseño e Implementación del Modelo 20
5.1. Preprocesamiento de datos 21
5.2. Implementación del modelo PointNet 21
5.3. Implementación de las técnicas de Data Augmentation 22
5.4. Separación del DataSet y Aplicación del Data Augmentation 26
5.5. Entrenamiento del modelo 27
6. Experimentos y Resultados 28
6.1. Métricas utilizadas para el análisis de los resultados 29
6.2. Experimento 1: Clasificación de Grano Grueso vs Clasificación de Grano Fino 30
6.3. Experimento 2: PointNet vs PointNet++ 32
6.4. Experimento 3: Aplicación de Data Augmentation para balanceo de DataSet 33
6.5. Experimento 4: Aplicación de Data Augmentation para aumento de DataSet 35
7. Conclusiones 35
8. Bibliografía 36

1. Introducción

1.1. Contexto

En la última década, el avance de la tecnología ha transformado nuestra forma de vivir y movernos por la ciudad gracias a sistemas como las smart cities o los coches con conducción autónoma. Para que estos servicios municipales y los vehículos autónomos funcionen correctamente, necesitan saber con precisión lo que hay a su alrededor. Ya no es suficiente con que detecten un obstáculo para no chocar; ahora es fundamental que sepan identificar qué objetos tienen delante, ya sea un banco, una papelera o una alcantarilla.



Para conseguir identificar objetos, el sensor LiDAR se ha convertido en la herramienta principal. A diferencia de las cámaras tradicionales, que tienen problemas cuando hay poca luz, sombras o mal tiempo, el LiDAR crea un mapa en tres dimensiones mediante ‘nubes de puntos’. Esto permite medir con más precisión el tamaño y la forma de los objetos, ofreciendo una información mucho más fiable del entorno [1].

El avance de la Inteligencia Artificial ha cambiado la forma de analizar estos datos en 3D. Modelos como PointNet han demostrado que es posible extraer información directamente de estas nubes de puntos. Sin embargo, estos sistemas tienen un problema importante: necesitan una cantidad enorme de datos, que deben ser antes ordenados y etiquetados por humanos para poder aprender.

Aqui es donde encontramos uno de los grandes problemas de la visión 3D en las ciudades. Conseguir datos con sensores LiDAR es difícil, y ordenarlos y ejecutarlos a mano requiere demasiado tiempo y dinero. Por eso, los conjuntos de datos de objetos reales que tenemos son pequeños y están muy descompensados: es muy fácil encontrar cientos de ejemplos de un banco común, pero si son objetos menos frecuentes es más difícil de encontrar y por lo tanto no tenemos tantos.

Esta falta de datos y esta diferencia de cantidad entre unos objetos y otros impide que el sistema pueda distinguir entre objetos muy parecidos. Es decir, es muy fácil enseñar a la tecnología a diferenciar un banco de una papelera, pero es mucho más difícil que distinga entre dos modelos distintos de papelera. Al tener tantos ejemplos de un tipo y tan pocos de otro, la inteligencia artificial termina por memorizar solo lo más común, perdiendo la capacidad de reconocer los objetos menos frecuentes o de los que hay pocas muestras.

Por todo ello, es necesario encontrar formas de aprovechar al máximo los datos que tenemos. En este contexto, las técnicas de Data Augmentation son la solución a este problema. El objetivo del proyecto es demostrar como el Data Augmentation hace que los modelos como PointNet sean más precisos al reconocer objetos reales de mobiliario urbano [2].

1.2. Planteamiento del Problema

Obtener y clasificar nubes de puntos en 3D en la ciudad es algo complejo y costoso. A diferencia de hacer fotos normales que se pueden sacar con la cámara de cualquier teléfono, usar sensores LiDAR requiere equipos más caros. Además, capturar esas imágenes requiere más tiempo que las fotos normales del móvil. También, ordenar y etiquetar los datos a mano en un trabajo bastante lento.

Debido a esto, la cantidad de datos de mobiliario urbano suele ser pequeña. Sin embargo, el tamaño no es el único problema ni el más grave; el verdadero obstáculo es que, en el mundo real, no hay la misma cantidad de todos los objetos. Por lo tanto, hay objetos que son menos comunes y de los que acabamos teniendo muy pocas muestras.

En cualquier ciudad, hay objetos que son mucho más comunes que otros. Al capturar datos, es normal capturar cientos de imágenes de un banco estándar, pero muy pocas de una papelera antigua o de diseño especial. Esto en inteligencia artificial se llama “distribución de cola larga”: unas pocas categorías representan la mayoría del conjunto de datos, mientras que muchos objetos apenas tienen ejemplo para que la maquina aprenda [3].

Este desequilibrio empeora la precisión de la inteligencia artificial. El PointNet es una herramienta muy potente, pero necesita una cantidad muy grande de datos para funcionar bien. Si la red se entrena con un DataSet muy desequilibrado, por ejemplo, con muchísimos bancos y muy pocas papeleras, es muy probable que la red neuronal tienda a decir únicamente el objeto más común. Así el modelo fallaría si se encuentra con una papelera, porque pensaría que es un banco. El modelo asume que todo lo que ve pertenece a la categoría que más veces ha visto.

Por culpa de este problema, la clasificación de grano fino (que el modelo no solo reconozca que es un “banco” sino que sepa especificar de que modelo de banco se trata) es muy difícil si solo usamos datos reales. Como tendremos muy pocos ejemplos de modelos muy específicos, hará que el modelo no sea capaz de identificarlos. Por lo tanto, es necesario que el DataSet este equilibrado.

En resumen, el proyecto se centra en solucionar el problema que genera la diferencia entre lo que necesita la inteligencia artificial (mucha cantidad de todo tipo de objetos) y lo que ofrece la realidad (pocas muestras y DataSet desequilibrados). Como es imposible salir a la calle y escanear cantidades tan grandes de objetos raros o poco frecuentes por falta de tiempo, el objetivo será encontrar y utilizar técnicas con las que podamos crear esos datos que faltan de manera artificial, sin capturarlos nosotros manualmente [4].

1.3. Objetivos

Para resolver el problema explicado anteriormente, el objetivo será utilizar técnicas de Data Augmentation que consigan que el modelo sea más preciso y no se limite a aprender solo los objetos más comunes.

Para conseguir eso, estas las tareas que se han hecho:

1. **Capturar y preparar el DataSet de objetos en 3D:** Salir a la calle para capturar nubes de puntos LiDAR de objetos de mobiliario urbano. Después, segmentar esas nubes de puntos para eliminar el ruido (quitar el suelo, paredes, cosas del entorno) y quedarnos únicamente con el objeto. Y por último el etiquetado y separación de los diferentes tipos de objetos capturados.

2. **Implementación del modelo de inteligencia artificial:** Desarrollar y comprobar la red neuronal PointNet, y entrenarla con los datos capturados, nubes de puntos originales. El objetivo es ver cuál es el resultado utilizando solamente las nubes originales capturadas, sin data augmentation.

3. **Encontrar los mejores parámetros para cada técnica de Data Augmentation:** Implementar varias técnicas de Data Augmentation y aplicarlas al conjunto de datos originales que ya tenemos. La idea es probar diferentes valores para los parámetros de cada técnica para poder encontrar el valor óptimo.

4. **Diseño del código para equilibrar y aumentar el tamaño del DataSet:** El objetivo aquí no es solo equilibrar el DataSet. También se puede aumentar el tamaño del DataSet con un multiplicador (x1, x2, x3, etc.), aplicando las técnicas de Data Augmentation.

5. **Análisis de los resultados:** Evaluar el resultado obtenido y comparar la precisión del modelo antes y después de implementar Data Augmentation

2. Presupuesto y Viabilidad Económica del Proyecto

Para analizar la viabilidad económica del proyecto, se ha hecho un presupuesto detallado de todos los gastos. Los costes se han separado en cuatro bloques principales: costes de personal, costes de hardware, costes de software y costes indirectos.

2.1. Costes de Personal

En este apartado se incluyen las horas de trabajo realizadas para hacer el proyecto.

- Ingeniero Junior: Se estima una dedicación total de 300 horas. El coste estimado para este perfil es de 10 €/hora.
- Director del Proyecto: Se estima una dedicación total de 30 horas. El coste estimado para este perfil es de 25 €/hora.

Perfil	Horas dedicadas	Coste por hora (€/hora)	Coste Total (€)
Ingeniero Junior	300	10 €	3000 €
Director	30	25 €	750 €
Coste Personal			**3750 €**

2.2. Costes de Hardware (Amortización)

Para la captura de datos hemos usado un iPad con sensor LiDAR, y para el entrenamiento del y realización de los experimentos hemos utilizado un ordenador portátil. Estos equipos se seguirán usando una vez terminado el proyecto, por lo tanto, no se incluye el precio de compra, sino que se calcula su amortización durante el tiempo de desarrollos.

Para el cálculo se estima una vida útil tecnológica de 4 años (48 meses) y un periodo de uso en este proyecto de 6 meses. La fórmula aplicada es: *(Coste Total / 48 meses) * 6 meses*.

Hardware	Precio (€)	Coste Mensual (€)	Coste Amortizado (€)
iPad	1000	20,83 €	125 €
Ordenador Portátil	1500	31,25 €	187,5 €
Coste Hardware			**312,5 €**

2.3. Costes de Software

La gran mayoría de software utilizado ha sido gratuito, por lo que los costes en estos casos son 0.

- **Polycam (Licencia):** Software usado para la captura y exportación de las nubes de puntos a formato .laz. Se ha utilizado para el cálculo el coste de una licencia anual.
- **Software Open Source:** También se ha utilizado muchos softwares de uso gratuito como CloudCompare para la segmentación, y librerías de Python para la creación del modelo.

Software	Coste (€)
Polycam (Licencia)	150 €
Software Open Source	0 €
Coste Software	**150 €**

2.4. Costes Indirectos

Aqui se incluyen los gastos de suministros utilizados durante los 6 meses de realización del proyecto, por ejemplo, consumo eléctrico, conexión a internet, etc.

Concepto	Coste Total (€)
Suministros (electricidad, internet, etc.)	150 €
Costes Indirectos	**150 €**

2.5. Presupuesto Total

Partida	Importe (€)
Costes de Personal	3750 €
Costes de Software	312,5 €
Costes de Hardware	150 €
Costes Indirectos	150 €
Base Imponible	**4362,5 €**
IVA (21%)	916,12 €
PRESUPUESTO TOTAL	**5278,62 €**

3. Marco Teórico y Estado del Arte

3.1. Procesamiento de Nubes de Puntos 3D

Primero vamos a explicar el formato en el que hemos capturado los objetos. Una nube de puntos es, un conjunto de puntos que representan un objeto. Esto se captura mediante sensores especiales, como la tecnología LiDAR. Para capturarlas nosotros hemos utilizado la aplicación Polycam con un iPad que contaba con un sensor LiDAR.

Desde un punto de vista más técnico, una nube de puntos es un conjunto de datos en un espacio de tres dimensiones (x, y, z). Cada punto representa una posición especifica en cada una de estas dimensiones.

Utilizar nubes de puntos es un cambio muy grande respecto a cómo se hacía antes. Durante mucho tiempo, la inteligencia artificial ha tenido mucho éxito analizando fotos normales (2D) porque en esa estructura, es mucho más fácil para el algoritmo. Sin embargo, los datos 3D son mucho más difíciles de analizar. Las nubes de puntos cumplen estas tres características:

1. **Datos desordenados:** A diferencia de una foto, donde los píxeles siguen un orden fijo, en una nube de puntos no hay un orden fijo. Una papelera 3D sigue siendo una papelera independientemente del que punto analicemos primero y cual el último. Por eso, al modelo de inteligencia artificial le debería dar igual en que orden recibe los datos.

2. **Los puntos están relacionados entre ellos:** Un punto por sí solo no aporta nada. Para reconocer un objeto hay que mirar cómo se organizan los puntos entre sí. Es decir, la forma que hacen esos puntos, si es la forma de un banco o una papelera. Si simplemente tenemos un punto, eso no nos dice nada.

3. **Reconocer giros y movimientos:** Un objeto 3D debe ser identificado sin importar hacia donde este mirando el objeto, porque la composición del objeto (posición de los puntos ente sí) es la misma.

Antes de que se crearan modelos de IA específicos para reconocer objetos 3D, lo que hicieron fue intentar adaptar los que ya existían para 2D. Se crearon estos dos modelos, aunque tenían algunos fallos:

1. **Voxelización (Cubos 3D):** Consiste crear cubos llamados Vóxeles . El problema era que requería mucha potencia en el ordenador. Para que el ordenador no se bloquee, los cubos eran grandes y se perdían los detalles pequeños de las imágenes.

2. **Proyección Multivista:** Este método lo que hace es sacar muchas fotos 2D del objeto 3D desde varios ángulos distintos. Esto hacía que no hiciera falta tanta potencia de ordenador como en la anterior, pero se perdía mucha información del objeto.

Esto demostró que utilizar para objetos 3D los mismos modelos que se utilizaban para las fotos 2D, no funcionaba bien. La solución fue crear un modelo específico para reconocer objetos en 3D, teniendo en cuenta las tres características mencionadas anteriormente. Por eso, se creó el modelo PointNet

3.2. Modelo PointNet

Como se ha explicado en el apartado anterior, era necesario crear un modelo que reconociera lob objetos 3D tal y como son, en lugar de transformarlo a partir de imágenes 2D como se hacía hasta ahora. Ese problema se consiguió solucionar con PointNet, el primer modelo de inteligencia artificial consiguió solucionar el problema de los datos desordenados. PointNet consiguió que diese igual en que orden se leían los datos, y por eso es la base de casi toda la tecnología 3D actualmente.

3.2.1. Funcionamiento del modelo PointNet

El objetivo es analizar el objeto sin importar el orden en el que se leen los datos. Para hacerlo, PointNet implementa una función matemática, anidando estas 3 funciones [(5)]:

1. **Analizar los puntos (h):** Coge un punto (P_x), lo estudia a fondo y convierte las 3 coordenadas (x, y, z) en 1024 características. Es como si sacara una ficha técnica muy detallada de cada uno de los puntos. Hace esto con todos los puntos de la nube.

2. **Función resumen o "Función simétrica" (g):** Aquí es donde se resuelve el problema de los datos desordenados. Una vez tenemos las 1024 características de cada punto, utiliza una función matemática (llamada Max Pooling) para quedarse con el valor más alto de cada una de las 1024 características. Da igual en que orden lea los puntos, si busca siempre el valor más alto para cada característica el resultado será siempre el mismo. Esto es lo que se llama función simétrica, quedarse con el resumen global del objeto.

3. **La decisión final (Y):** Ahora que la red tiene ese resumen de 1024 características, decide a que objeto corresponden esas características.

La función matematica que define esto es la siguiente, donde xi es cada uno de los puntos:

$$f(x_1, \ldots, x_n) \approx \gamma \left(\max_{i=1,\ldots,n} \{h(x_i)\} \right)$$

3.2.2. Operación Max Pooling y su impacto en el Data Augmentation

PointNet es conocida por ser una arquitectura muy robusta y resistente al ruido y a las pequeñas transformaciones. El principal motivo de esto es la función Max Pooling que, como hemos explicado arriba, es la genera el resumen general del objeto, quedándose tan solo con 1024 características.

Una vez que se han analizado cada uno de los puntos por separado y se le ha aplicado a cada uno de ellos 1024 características, la función Max Pooling examina cada una de las características y se queda, de entre todos los puntos, el valor más grande. De forma que, si tenemos una nube de 2048 puntos y a cada uno le aplicamos 1024 características, nos quedarían 2048x1024 características en total. Al aplicar la función Max Pooling, convertimos la matriz de puntos en tan solo un vector de 1024 características globales.

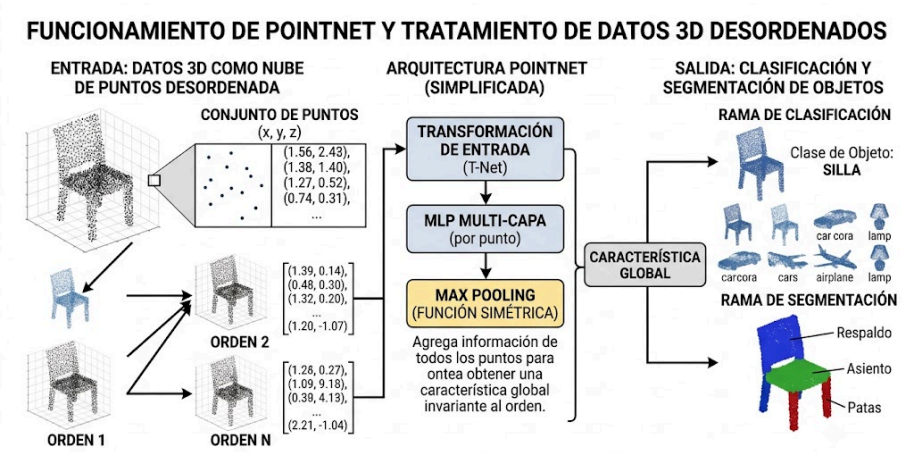


Figura X: PointNet procesa nubes de puntos 3D desordenadas sin necesidad de alineación o conversión previa. Su arquitectura, que incluye una función simétrica (Max Pooling), permite extraer una característica global invariante al orden de los puntos, fundamental para tareas de clasificación y segmentación precisa de objetos 3D.

3.2.3. Redes de Alineación (T-Nets)

Para que el sistema reconozca un banco independientemente de si se ha escandeado de frente, de un lado o de otro, PointNet utiliza un módulo llamado T-Net (Red de Transformación). T-Net analiza los puntos para calcular la orientación del objeto y después, utiliza una función matemática que lo que hace es girar la nube de puntos y ponerla en una posición estándar. Gracias a esto el modelo siempre analiza el objeto desde la misma dirección.

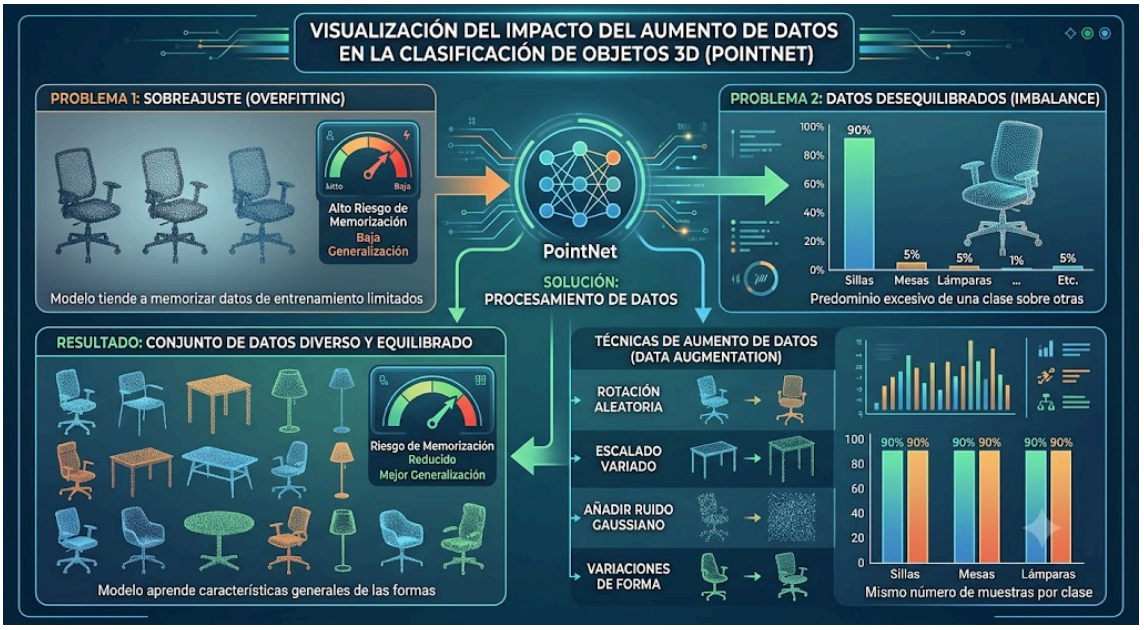
3.3. Data Augmentation en Nubes de Puntos LiDAR

A pesar de que la red PointNet es muy buena para analizar y predecir objetos 3D, hay un riesgo de que el modelo memorice los datos de memoria en lugar de aprender a identificarlos (Overfitting). Si esto pasa y el modelo simplemente memoriza los datos que le pasamos para entrenar, al ver un objeto diferente no va a saber identificarlo.

Además, existe también el problema de tener unos datos desequilibrados, es decir, una cantidad muy grande de datos de un objeto, pero muy pocos datos de otro objeto. Por ejemplo, si el 99% de nuestros datos son bancos, el modelo va a aprender a decir solamente “banco”. De forma que, si le pasamos otro objeto distinto, lo va a identificar como un banco, aunque no lo sea.

Para solucionar estos dos problemas, utilizaremos el Aumento de Datos (Data Augmentation) para aumentar artificialmente la cantidad de datos que tenemos. Su objetivo será ampliar la cantidad de situaciones y características que el objeto puede tener, para obligar al modelo a aprender la forma del objeto real en lugar de memorizarlo. Además, se utilizará también para crear un DataSet equilibrado,

aumentando la cantidad de los aquellos objetos de los que disponemos de menos muestras [6].



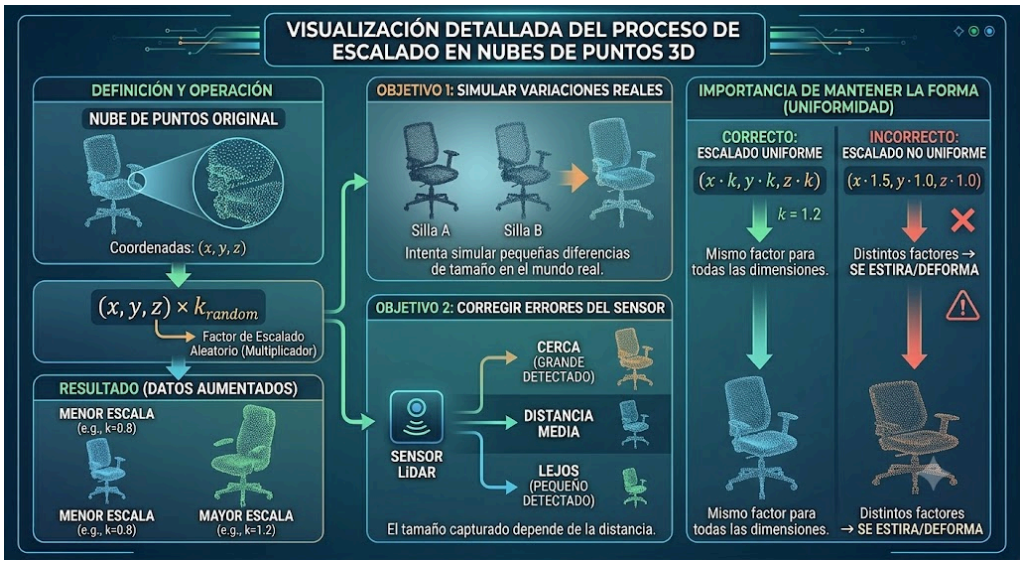
3.4. Técnicas de Data Augmentation

Las técnicas implementadas y analizadas en este proyecto son las siguientes:

1. **Rotación (Rotation):** Lo que busca es rotar el objeto para que el modelo aprenda el objeto independientemente del lado desde el que lo mira. Sin embargo, en el contexto del mobiliario urbano no podemos girar en objeto en cualquier dirección. Una rotación sobre los tres ejes (x, y, z) simultáneamente no tendría ningún sentido, porque los bancos o las papeleras siempre van a estar de pie sobre el suelo. Por lo tanto, no tiene sentido tratar de entrenar al modelo para que reconozca un banco boca abajo. Por lo tanto, la rotación óptima será siempre sobre el eje vertical (eje Z), para simular el mismo objeto visto desde diferentes lados [7].



2. **Escalado (Scale):** Esta técnica multiplica las coordenadas de cada punto por un número aleatorio, haciendo el objeto más grande o más pequeño. Sus objetivos son dos: por una parte, intenta simular las pequeñas diferencias de tamaño que puede haber entre dos objetos iguales en el mundo real; por otra parte, corrige posibles errores de medición del sensor y de la captura de datos, porque depende de la distancia desde la que se capture el objeto su tamaño puede variar significativamente. Para que el objeto siga teniendo la misma forma, el escalado se hace en las tres dimensiones, para evitar que el objeto se estire o se deforme.



3. **Ruido Aleatorio (Jittering):** Esta técnica sirve para que el modelo pueda identificar pequeñas imperfecciones a la hora de capturar el objeto o por errores del sensor. Consiste en mover cada uno de los puntos un poco. Esto se hace mediante el parámetro sigma (σ), que representa la desviación estándar utilizada para generar ruido. Cuanto mayor sea este parámetro, más se moverán los puntos de sitio. Para que esta técnica sea útil el ruido debe ser suficientemente grande para modificar el objeto significativamente y obligar a la red a aprender en lugar de memorizar. Sin embargo, debe tener un límite para evitar que el objeto cambie de forma por completo y visualmente deje de parecer lo que realmente es, porque confundiríamos al modelo.



4. **Eliminación de puntos (Dropout):** En la calle, los objetos casi nunca se escanean perfectos. Puede haber obstáculos (peatones, pivotes, etc.) que tapan parte del objeto, o pueden haber sido escaneados desde lejos y el sensor capta menos puntos. Para que el modelo sea capaz de identificar esos objetos también, se utiliza Dropout. Esta técnica lo que hace es eliminar aleatoriamente un porcentaje determinado de puntos.



4. Creación del Conjunto de Datos

4.1. Captura de datos

Para poder entrenar un modelo de inteligencia artificial que reconozca objetos 3D de mobiliario urbano, lo primero que debemos hacer es conseguir los datos que vamos a utilizar. En lugar de usar datos genéricos descargados de internet, en este proyecto hemos realizado nuestra propia recogida de datos en la calle. Esto asegura que los datos simulen los tipos de datos que podemos encontrarnos en una ciudad española actual.

Entorno y lugar

La captura de datos se hizo en la ciudad de Vitoria-Gasteiz (Álava). Esta ciudad cuenta con muchos tipos de objetos de mobiliario urbano de diferentes modelos, lo cual es justo lo que necesitamos para poner a prueba la utilidad real del Data Augmentation.

Hardware y Software utilizados

Tradicionalmente, para poder capturar nubes de puntos era necesario disponer de unas herramientas extremadamente caras. Sin embargo, esto hoy en día a cambiado e incluso algunos móviles iPhone tienen un escáner LiDAR integrado. Estudios recientes han demostrado que el escáner LiDAR de estos dispositivos, basado en la tecnología Direct Time-of-Flight (dToF), ofrece una precisión y densidad de puntos más que suficiente para la reconstrucción de objetos a corta y media distancia [(8)].

Para la captura de datos de este proyecto he utilizado un iPad que contaba con un escáner LiDAR. En cuanto al software, se ha utilizado la aplicación Polycam, la cual permite escanear el objeto de forma bastante precisa.

Escaneo de objetos

Para asegurar que los objetos se vieran con buena calidad desde todos los ángulos y no hubiese huecos de información, se hizo lo siguiente al escanear el mobiliario urbano:

- **Rodear el objeto:** Se rodeó cada objeto (banco, papelera, etc.) con el escáner LiDAR. Esto se hizo para capturar y poder ver el objeto desde todos los ángulos y evitar que haya algunas partes que tapen otras zonas y se queden sin escanear.

- **Cambio de altura:** Durante el recorrido, se iba variando la inclinación del iPad para capturar también la parte de arriba y la de abajo.

- **Limpieza de escena:** Para evitar que algunas partes se quedasen sin escanear, tratamos de evitar que durante el escaneo hubiese obstáculos de por medio que tapasen partes del objeto.

El resultado de todo este proceso fue la obtención de cientos de nubes de puntos (exportados en formato .laz) que representaban el mobiliario urbano de Vitoria.

4.2. Segmentación y etiquetado

Las nubes de puntos capturadas contienen siempre una cantidad bastante grande de ruido e información que no queremos. Es decir, cada archivo exportamos en formato .laz (LASzip), no incluye solo el objeto, sino que también puede incluir cosas como la acera, la hierba, la fachada de los edificios cercanos, etc. Esto confundiría a nuestro modelo a la hora de entrenarlo.

Por lo tanto, tuve que hacer una segmentación manual para eliminar ese ruido innecesario. Para hacerlo, utilice CloudCompare, un software de código abierto muy útil para el procesamiento de nubes de puntos 3D [(9)].

Una vez segmentado, era necesario organizar los objetos de una forma que después fuese fácil de cargar mediante scripts de Python. Por ello, los organice en directorios que representaban la clase o categoría general (bancos, papeleras, alcantarillas, buzones, contenedores, etc.). Para poder hacer después más fácil el análisis de grano fino, guarde los archivos especificando el modelo específico del objeto. Cada nube de puntos segmentada fue guardada dentro de la carpeta correspondiente a su clase y con el siguiente nombre del archivo: **{fecha de captura}_{objeto}_{modelo}_segmented.laz**

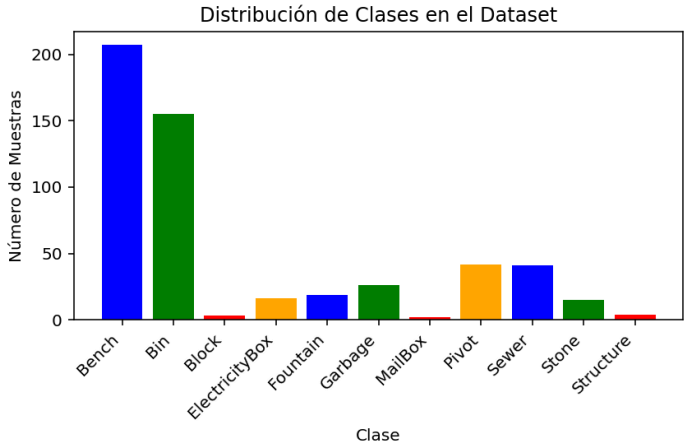
Este nombre no es solo para organizar carpetas, sino que facilitara la utilización de los datos para el entrenamiento del modelo. Gracias a esta estructura, el sistema de carga de datos será capaz de identificar tanto la categoría general (Banco) como el modelo (B501). Esto permite realizar experimentos de clasificación de grano grueso y de grano fino.

4.3. Análisis del DataSet

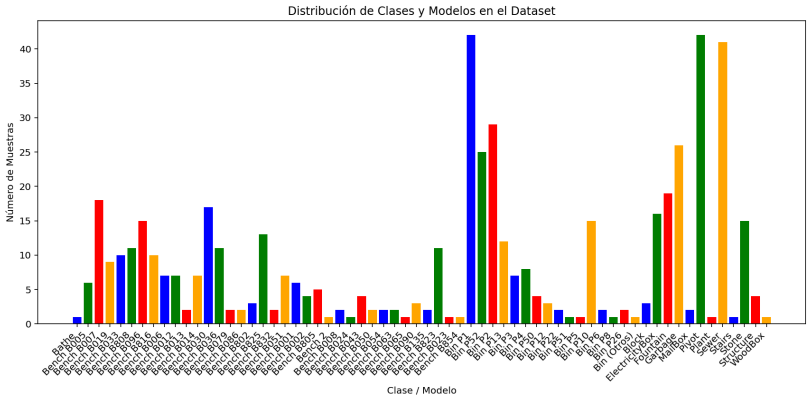
Una vez tenemos el DataSet capturado, segmentado y etiquetado, podemos hacer un análisis estadístico de los datos. Con este análisis podemos ver claramente el problema del desbalanceo que vimos anteriormente (apartado 1.2).

Vamos a separar el DataSet para analizarlo desde dos perspectivas diferentes:

1. **Clasificación de Grano Grueso (Por Clases):** Al agrupar los archivos en función únicamente de la clase o tipo de objeto al que pertenece, se ve claramente como el dataset está completamente desbalanceado. La cantidad de bancos y papeleras es mucho mayor que la de cualquier otro objeto. Además, hay algunas clases como Block, MailBox y Structure. Sin embargo, hay que tener en cuenta que la clasificación de grano grueso es bastante mas sencilla que la de grueso fino. Por ejemplo, diferenciar entre un banco y una papelera es más fácil que saber reconocer los distintos modelos de bancos.



2. **Clasificación de Grano Fino (Por Modelos):** Aqui es donde está la mayor dificultad para nuestro modelo, saber reconocer los distintos tipos de papeleras y bancos que existen, teniendo en cuenta que muchos de ellos son bastante parecidos entre sí. Si observamos el gráfico, aquí también hay una diferencia bastante en cuanto a la distribución. Sin embargo, al haber separado los dos tipos de objetos con mayor volumen de datos en diferentes modelos, la diferencia no es tan exagerada, aunque evidentemente sigue siendo un problema que debemos resolver.



La distribución de los datos mostrados en el gráfico muestra algo muy común en el mundo real: el desbalanceo de datos. Esto significa que unos pocos modelos de bancos y papeleras son muy abundantes en Vitoria-Gasteiz y aparecen mucho. En cambio, otros modelos, probablemente aquellos que son más antiguos y ya no se fabrican, son muy escasos y tan solo tienen dos o tres muestras.

Entrenar la red neuronal con estos datos tal cual, provocaría que el modelo aprendiera a reconocer solamente los objetos más comunes, ignorando el resto de los objetos [(10)]. Por ello, es imprescindible utilizar el Data Augmentation para equilibrar artificialmente estas diferencias sin tener que volver a la calle a buscar más datos.

4.4. Partición del Dataset

Este es el último paso en la preparación de los datos antes de entrenar la inteligencia artificial. Consiste en la división del conjunto de datos tres grupos: entrenamiento,

validación y test. Los datos de entrenamiento sirven para enseñar al modelo como es cada objeto y que así pueda aprender a identificarlos, después se le hace al modelo una prueba con los datos de validación para verificar si sigue aprendiendo. Finalmente, cuando el modelo alcanza sus mejores resultados en validación se utiliza ese modelo para hacer un examen final con los datos de test. Esta separación es fundamental para medir la capacidad real del modelo de identificar objetos que no ha visto antes.

Para este trabajo he hecho una división estándar del 70% para el conjunto de entrenamiento, 15% para el de validación y 15% para el conjunto de prueba (test) [11].

El aislamiento total de los test

Es muy importante tener en cuenta la siguiente regla: el conjunto de prueba del 15% y el otro 15% de validación son estrictamente inmutables.

Esto quiere decir que las técnicas de Data Augmentation que hemos visto en el apartado 2.3 tanto para balanceo como para multiplicación del DataSet, se aplican solamente al 70% correspondiente a los datos de entrenamiento. No se pueden utilizar las muestras generadas artificialmente para evaluar la precisión del modelo. Hacer esto crearía un escenario de evaluación que no sería real, porque los datos de los test compartirían las mismas variaciones que los datos utilizados para entrenar. Es decir, el 30% del dataset dedicado al test y validación actúan como un entorno de “mundo real” puro, y debe estar compuesto solamente por datos reales.

5. Diseño e Implementación del Modelo

5.1. Preprocesamiento de datos

Antes de empezar a entrenar el modelo, debemos adaptar los datos que tenemos a lo que espera obtener el modelo para poder ahorrarnos tiempo de cómputo. Esto se ha implementado mediante el script preprocesamiento_datos.py, el cual hace lo siguiente:

1. **Cambio de formato a los archivos:** Como hemos dicho anteriormente, los archivos originales está en formato LASzip (.laz). Este script cambia el formato de esto archivos a el formato binario de NumPy (.npy), lo cual es necesario para ahorrarnos tiempo de cómputo y que el entrenamiento sea más rápido. Durante el entrenamiento, el modelo va a necesitar iterar sobre el conjunto de datos varias veces. LASzip (.laz) son archivos comprimidos y necesitan ser descomprimidos en cada lectura, lo cual provocaría que al modelo le costase mucho más tiempo en leer los archivos y por lo tanto el entrenamiento tardaría demasiado. Utilizar archivos .npy nos permite realizar esa lectura mucho más rápido.

2. **Centrar las nubes de puntos:** Para facilitar el reconocimiento de los objetos, hemos colocado todos los objetos en la misma coordenada inicial. Es decir, he centrado el objeto en la coordenada (0, 0, 0). Además, he hecho que la escala sea de radio 1 para que todas tengan el mismo tamaño.

3. **Dimensión fija para los puntos:** Es necesario que todas las matrices de puntos tengan las mismas dimensiones para que el modelo pueda compararlas. Por eso, he fijado el número de puntos que queremos que tenga la nube. Es una variable en el código, por lo tanto, se puede cambiar y poner la que queramos. Para este proyecto he utilizado nubes de 2048 puntos.

4. **Almacenamiento y guardado:** Una vez hecho todo esto, el script genera una copia de todos los datos preprocesados y los guarda en la carpeta processed_data

5.2. Implementación del modelo PointNet

Para experimentar con la utilización de Data Augmentation, he implantado el modelo PointNet utilizando PyTorch. Esta es una librería de Python y es uno de los sistemas de aprendizaje profundo (Deep Learning) más utilizados. Además, permite aprovechar al máximo la potencia de las tarjetas gráficas (GPU) para hacer que el entrenamiento sea mucho más rápido.

El modelo que he implementado es una adaptación de la versión Vanilla de PointNet. El código es una adaptación de otra implementación sacada de un repositorio de GitHub[12]. El modelo funciona de la siguiente forma y los datos pasan por las siguientes 3 fases a lo largo del código, que son las mismas que vimos en el apartado 2.2.:

1. **Extraer las Características Locales (Feature Extractor):** El proceso comienza recibiendo (B, 3, N) donde B es el lote (Batch) de nubes de puntos, 3

es la cantidad de dimensiones (x, y, z) y N es la cantidad de puntos que tiene cada una de las nubes (en nuestro caso 2048). Como los puntos no tienen un orden fijo, el modelo los analiza uno a uno individualmente sin importarle el orden. Para ello, utiliza una operación especial llamada convoluciones de tamaño 1 que funciona de la siguiente manera: pasa las 3 coordenadas a un espacio con más detalle de 64, después pasa de esos 64 a 128, y finalmente a 1024 características por punto.

```
# Análisis de los puntos individualmente
x = F.relu(self.bn1(self.conv1(x))) # Salida: (Batch, 64, N)
x = F.relu(self.bn2(self.conv2(x))) # Salida: (Batch, 128, N)
x = F.relu(self.bn3(self.conv3(x))) # Salida: (Batch, 1024, N)
```

2. **Función Simétrica (Max Pooling):** Tras la primera fase, el modelo tiene cada punto en un espacio de muchas dimensiones (B, 1024, N). En este punto se aplica la operación matemática vista en el apartado 2.2: Max Pooling.

   Mediante la función torch.max, el modelo analiza las 1024 características y selecciona solamente el valor máximo de cada una de ellas para todos los puntos. De esta forma, se queda solamente con el vector global de 1024 características que resume todo el objeto (B, 1024). Al quedarse solo con los valores máximos, el modelo identifica los puntos críticos y descarta lo demás. Además, con este método logramos que no importe el orden en el que se leen los puntos

```
# Buscamos el valor máximo entre todos los puntos.
# Esto hace que no importe el orden de los puntos.
x = torch.max(x, 2, keepdim=True)[0]
x = x.view(-1, 1024) #Salida: (Batch, 1024)
```

3. **Clasificación:** Una vez obtenido el vector de 1024 características que resume el objeto completo, pasa por una serie de capas que va reduciendo el número de características hasta llegar a la respuesta final. Pasa de 1024 a 512, después a 256. Antes de la clasificación final, el sistema pasa por una capa de Dropout (apagado aleatorio). Esto apaga al azar el 30% de las neuronas durante el entrenamiento, lo que hace que la red sea más robusta, evitando que aprenda los datos de memoria (overfitting)[13].

```
x = F.relu(self.bn4(self.fc1(x))) # De 1024 a 512
x = F.relu(self.bn5(self.fc2(x))) # De 512 a 256
x = self.dropout(x) # Capa de dropout (apaga neuronas)
x = self.fc3(x) # De 256 a "num_classes"
```

5.3. Implementación de las técnicas de Data Augmentation

Para solucionar los problemas de escasez de datos y el desbalance entre clases que se han especificado en los apartados anteriores, he implementado el script

augmentations.py para aplicar diferentes técnicas de Data Augmentation. He separado las diferentes técnicas por funciones con diferentes parámetros para ajustar el impacto de la técnica. Estas técnicas se aplican sobre el conjunto de datos originales de entrenamiento.

Al ejecutar el código genera una nueva carpeta con el nuevo dataset con los datos artificiales ya incluidos, pero antes de ello hay que ajustar algunas variables:

1. **DIRECTORIO DE ENTRADA:** Es la ruta donde se encuentran los datos originales a los que le vamos a aplicar Data Augmentation. Hay que poner la ruta donde se encuentran solo los datos dedicados al entrenamiento, como hemos explicado anteriormente.

   ```
   DIRECTORIO_ENTRADA =
   "../../DataSet/Separados/Por_Modelos/Train_Original"
   ```

2. **DIRECTORIO DE SALIDA:** La ruta donde queremos que se guarde el nuevo DataSet. Este nuevo DataSet contendrá los datos originales además de los nuevos datos generados artíficamente.

   ```
   DIRECTORIO_SALIDA =
   "../../DataSet/Separados/Por_Modelos/Augmented_Data/"
   ```

3. **MULTIPLICADOR:** Es el numero por el que queremos aumentar el tamaño del DataSet. Se coge la clase con más muestras y se multiplica por el multiplicador, y después se aplica Data Augmentation hasta llegar a ese número con todas las clases.

   ```
   MULTIPLICADOR_DATASET = 5
   ```

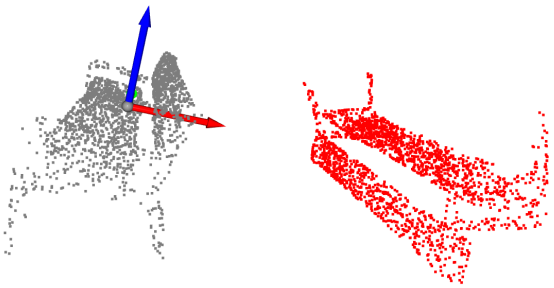
4. **TECNICAS_AUGMENTATION:** Especificamos el nombre de la técnica o las técnicas que queremos utilizar.

   ```
   TECNICAS_AUGMENTATION = ['rotate', 'jitter', 'dropout']
   ```

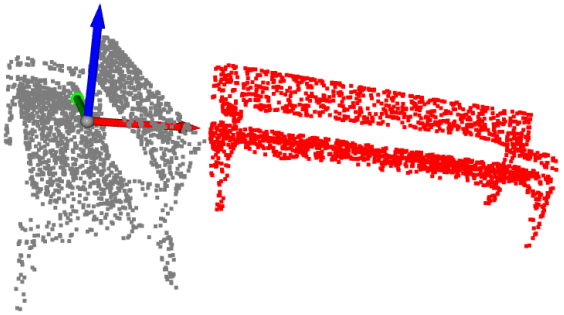
Implementación de las técnicas

Con el script verificar_augmentations.py podemos ver de forma visual la deformación que hacen las técnicas en cada uno de los objetos. Estas son las técnicas que he implementado y la imagen generada para verlo de forma visual:

1. **Rotate:** Esta técnica rota el objeto en el eje o ejes que le especifiquemos por parámetro. Contiene dos parámetros: axes (array para especificar los ejes en los que debe rotar) y max_angle_deg (grados máximos que puede rotar el objeto).



Como podemos ver hemos rotado el banco, se ha puesto al revés. Sin embargo, en nuestro caso queremos que el banco rote en el eje z para que se mantenga de pie, porque en el contexto del mobiliario urbano, el modelo nunca se va a encontrar un banco dado la vuelta de esa forma. Por lo tanto, podemos cambiar el parámetro para que rote en el eje z.



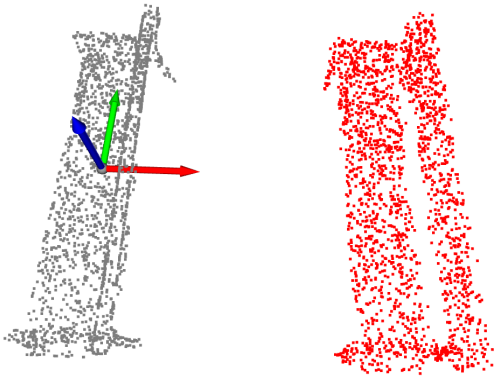
2. **Jittering:** Añade ruido a los puntos, es decir, los mueve todos un poquito desde su posición original. Simula pequeñas imperfecciones en la captura de datos. Utiliza dos parámetros: sigma (Intensidad del ruido. Representa la desviación estándar, cuanto más alta sea más se moverán los puntos) y clip (Es el límite de seguridad. Es el valor máximo que un punto se puede mover).

   El código es una adaptación de un repositorio público de GitHub[14]. Sin embargo, he tenido que cambiar ciertas cosas. El código del repositorio mencionado recibe 3 entradas y en mi caso lo he hecho con 2:

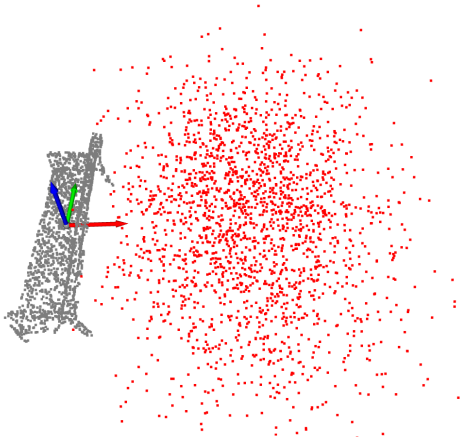
   - B – Batch (Cantidad de nubes de puntos en un mismo lote)
   - N – Número total de puntos
   - C – Coordenadas (En mi caso 3: x, y, z)

   En ese código se cargan los datos limpios, se crea el lote (Batch) gigante de, por ejemplo, 32 nubes, y se aplica la fórmula matemática a las 32 nubes a la vez.

   En mi código (2 entradas, sin batch): Se coge un archivo LiDAR, se cargar la nube de puntos, se le aplica el ruido (función jittering) y luego se devuelve la imagen ya modificada para formar el lote después. La creación del lote se hará a la hora de entrenarlo.

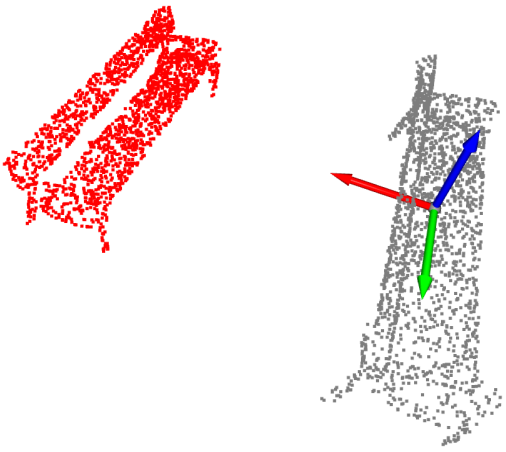


Apenas se puede ver la diferencia a simple vista, pero realmente los puntos están un poco movidos. Esto es porque el valor de sigma es muy pequeño y hace que se muevan tan poco que apenas se aprecia. Para comprobar que funciona bien he subido el valor de sigma a 0.5 y el de clip a 1 para poder ver la diferencia, aunque luego esto tiene que bajar cuando lo vaya a utilizar para entrenar porque podría confundir al modelo.



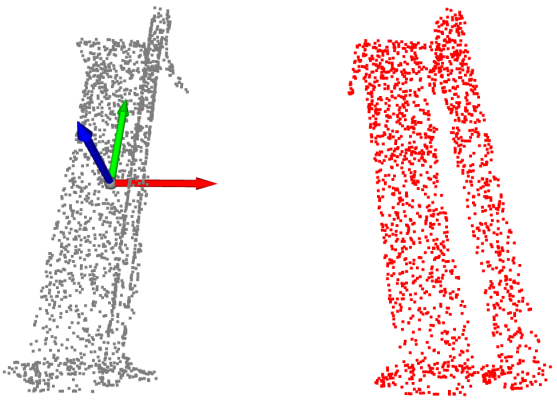
Como vemos se han movido tanto los puntos que ya ni siquiera se ve que es un banco. Por eso, al entrenar el modelo, el valor de sigma y clip deben ser muy pequeños para que simplemente añada "ruido" y no cambie completamente la forma del banco. Al menos esto nos sirve para comprobar de forma visual el funcionamiento de Jittering.

3. **Scale:** Escala el objeto de tamaño. Lo hace un poco más pequeño o un poco más grande que el original. Recibe un parámetro de variación que representa el porcentaje máximo que puede variar. Aquí en este ejemplo, el banco modificado se ha hecho muchísimo más pequeño que el banco original.

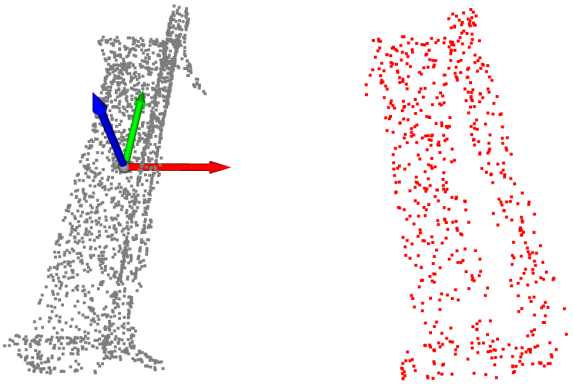


4. **Dropout:** Elimina puntos aleatorios de la nube. Sin embargo, la nube es de tamaño fijo (por ejemplo, 2048 puntos) para que la matriz donde están puestas las coordenadas sea del mismo tamaño en todas las imágenes. Por eso no se pueden eliminar puntos directamente y lo que se hace es mandarlos a todos a la misma coordenada. De esta forma en la matriz va a haber varias entradas que apunten a la misma coordenada y el modelo lo interpreta como un único punto, pero el tamaño de la matriz sigue siendo el mismo.

   La cantidad de puntos que se "eliminan" depende del parámetro dropout_radio. En estos momentos está en 0,125. Es decir, se eliminan el 12,5% de los puntos.



   Al igual que con el jittering, apenas se aprecia la diferencia. Para comprobar que funciona y verlo de forma visual, he subido valor de dropout_ratio a 0,7 (se borran el 70% de puntos).



5.4. Separación del DataSet y Aplicación del Data Augmentation

Para la división del Dataset lo he hecho mediante el script SepararDataset.py. Este código lo que hace es cortar el conjunto de datos en tres partes distintas de manera aleatoria, pero manteniendo siempre unos porcentajes de cada modelo en todas las carpetas.

En primer lugar, el algoritmo extrae del nombre del archivo la clase y el modelo del objeto y agrupa todos los datos por modelos diferentes. Luego, utiliza un filtro para descartar aquellos grupos que contengan menos de 3 muestras originales. Esto se hace para asegurarnos de tener, al menos, 1 muestra en cada una de las particiones (entrenamiento, validación y test).

Después, las muestras que han quedado se dividen aleatoriamente en tres grupos, según los siguientes porcentajes:

1. **Entrenamiento (Train):** Aproximadamente el 70% de los datos. Estos se destinarán a conseguir que el modelo aprenda a identificar.

2. **Validación (Val):** Un 15% de los datos. Se utilizan para comprobar si el modelo sigue aprendiendo o si está empezando a memorizar (overfitting). En el momento en que se detecta que está memorizando se detiene el proceso.

3. **Prueba (Test):** Aqui van los datos restantes, un 15% más o menos. Se usan para hacerle al modelo la prueba final con datos que no ha visto nunca, una vez el proceso se haya detenido.

Aplicación del Data Augmentation

Una vez tenemos los datos separados en los tres grupos (Train, Val, Test) podemos ejecutar el script augmentations.py visto anteriormente.

Es muy importante destacar que el Data Augmentation se debe ejecutar única y exclusivamente sobre el conjunto de entrenamiento (train). Estas técnicas se utilizan para crear muestras artificiales y por así expander la cantidad de datos de aprendizaje, forzando al modelo a aprender objetos de diferentes estilos y con posibles defectos en la

captura de esos datos. Los datos de validación y prueba se utilizan para evaluar el modelo y deben permanecer intactas, solamente con muestras originales.

Alterar algorítmicamente el subconjunto de Test supondría una violación del principio de evaluación independiente, introduciendo una fuga de datos (Data Leakage) que contaminaría las métricas finales al evaluar a la red neuronal sobre distribuciones sintéticas que ella misma ya ha aprendido a tolerar durante el entrenamiento [15]. De este modo, la separación de datos garantiza que el modelo será evaluado solo con muestras reales.

5.5. Entrenamiento del modelo

Una vez hemos terminado de hacer todo lo anterior ya podemos ejecutar el código para entrenar el modelo.

Configuración del Entrenamiento y variables

Para entrenar el modelo utilizaremos el script entrenamiento.py. En este script debemos ajustar la dirección donde se encuentran los datos que vamos a utilizar. Es decir, las carpetas que conseguimos con al separar el dataset (con SepararDataset.py) de Train, Val y Test.

La red neuronal no procesa todos los datos a la vez, los procesa en pequeños grupos. El tamaño de cada grupo (Batch Size) está puesto en 32 muestras; esta cantidad es el equilibrio perfecto que se suele utilizar, porque así la tarjeta gráfica (GPU) puede analizar muchos objetos a la vez, pero sin que llegue a colapsar.

Además, el modelo utiliza también una variable llamada Learning Rate. Es la rapidez a la que aprende la IA. Si el valor es muy alto, el modelo aprenderá rápido, pero corre el riesgo de saltarse o pasar por alto datos importantes, por lo que el resultado puede ser peor. Si el valor es demasiado bajo, sucede lo contrario, aprende bien, pero podría tardar mucho en terminar de ejecutarse. Nuestro objetivo es encontrar un punto intermedio.

Por último, la última variable que podemos ajustar son los Epoch. Esto es la cantidad de vueltas que el modelo le da al temario. La idea sería poner un número alto para poder aprender mejor, pero si ponemos demasiados Epoch, corremos el riesgo de que el modelo comience a memorizar los datos en lugar de aprender. Sin embargo, en nuestro caso, vamos a utilizar un Epoch excesivamente alto, ya que utilizaremos los datos de validación (los conseguimos al separar el dataset) para asegurarnos de que no está memorizando. En el momento en el que detectemos que está memorizando, el proceso se para (Early Stopping).

Validación y *Early Stopping*

Como hemos dicho antes, para evitar que el modelo memorice los datos y el ruido que les hemos incluido mediante el Data Augmentation, la cantidad de veces que se ejecuta el entrenamiento (Epoch) depende del resultado de la prueba de validación.

Cada vez que se da una vuelta al temario, se le hace al modelo una prueba utilizando los datos de validación (los datos de test aún no se usan, se guardan para el final para asegurarnos de que no los ha visto nunca hasta el momento del test final). Se considera

que el modelo está aprendiendo todavía si el resultado de ese test de validación es mejor que el anterior. En el momento de que el resultado de los test de validación no haya mejorado en las últimas ejecuciones, el proceso se detiene.

La cantidad de veces que debe estar el modelo sin mejorar el resultado de validación antes de que salte el Early Stopping se define en una variable en el código. En mi caso he utilizado 10.

Una vez que el proceso se ha parado, se coge el modelo que mejor resultado ha obtenido en la prueba de validación y realiza la prueba final, utilizando los datos de Test (datos que son nuevos para él y que no ha visto nunca). Al no haber visto estos datos nunca, el resultado que extraigamos de esa prueba será la precisión real del modelo identificando objetos de mobiliario urbano del mundo real[16].

Automatización del proceso

Como para hacer este proyecto necesitamos hacer muchas pruebas con muchas técnicas y muchos parámetros diferentes, he creado un script de entrenamiento automático. Utiliza PyTorch para el aprendizaje y Pandas para organizar los datos. El programa ejecuta el entrenamiento con diferentes parámetros y registra los resultados de cada uno de forma automática.

Para hacer esto le pasamos un directorio como datos de entrada donde se encontrar todas las carpetas de una técnica en específico, cada una de ellas con datos de diferentes parametros. Finalmente nos devuelve el resultado en forma de tabla. Para que funcione, las carpetas con todos los datos las debemos guardar dándoles el siguiente nombre: **{nombre que queramos}{Fila}.{Columna}**

6. Experimentos y Resultados

6.1. Métricas utilizadas para el análisis de los resultados

Para evaluar de forma correcta los resultados obtenidos del entrenamiento, se ha utilizado como medida el F1-Score Macro. En el mundo real, suele existir cierto desbalanceo en el Dataset (objetos con muchas muestras y otros con muy pocas muestras). Este desbalanceo hace que la métrica Accuracy, es decir, ver el porcentaje de aciertos en general, no sea del todo fiable. El modelo aprenderá a analizar aquellas clases de las que se dispone muchos datos, dejando las demás de lado, y esto va a hacer que el porcentaje de Accuracy sea muy elevado.

Por ejemplo, supongamos que queremos crear una inteligencia artificial que detecte si un paciente tiene cáncer o no. La cantidad de gente que no tiene cáncer en el mundo real es mucho mayor grande que los que si tienen, por lo tanto, es muy probable que nuestro modelo aprenda a decir siempre que el paciente no tiene cáncer, aunque si lo tenga. El Accuracy del modelo será probablemente superior al 99%, pero no sería un modelo útil en absoluto [17].

Esta es la razón por la que vamos a analizar el modelo en base al F1-Score Macro. Esta métrica se calcula a través de la librería de Python Scikit-Learn. Sin embargo, el sistema también calcula otras métricas. Estas son todas las métricas que nos calcula nuestro modelo:

1. **Precisión (Accuracy):** Esto es el porcentaje de acierto que ha logrado el modelo. Es decir, si de 10 imágenes en el test final, acierta 5, el resultado será 50%. Matemáticamente:

$$\text{Accuracy} = \frac{\text{Predicciones Correctas}}{\text{Total de Predicciones}} \times 100$$

2. **Loss:** Es la seguridad con la que identifica un objeto, es decir, si lo acierta por suerte o porque realmente lo ha identificado correctamente. Al final de la función, cuando el modelo debe decidir de que objeto se trata, piensa cual es el más probable según el análisis que ha hecho. Por ejemplo, supongamos que el resultado final es este: 27% banco, 25% silla, 25% papelera y 23% alcantarilla. En este caso el modelo dirá que es un banco. Puede que acierte, pero por pura suerte, no porque tenga la certeza de que eso era un banco. Por lo tanto, en este caso el Loss sería muy alto.

3. **F1-Score Macro:** Es la métrica más fiable, y es en la que nos fijaremos después para analizar los resultados. Es la media armónica entre la Precisión (de lo que predice como clase X, cuánto ha acertado) y el Recall (de todas las clases X reales que había, cuántas ha conseguido encontrar). Además, como es “Macro”, calcula el F1-Score para cada una de las clases que tenemos por separado y luego hace una media entre los F1-Score calculados de cada una.

Matemáticamente: Primero se calculan los Verdaderos Positivos (TP), Falsos Positivos (FP) y Falsos Negativos (FN) para cada clase i. Y se calcula el F1-Score de cada clase:

$$\text{Precision}_i = \frac{TP_i}{TP_i + FP_i}$$

$$\text{Recall}_i = \frac{TP_i}{TP_i + FN_i}$$

$$F1_i = 2 \times \frac{\text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}$$

Y después, como es "Macro", hace la media de todos los F1:

$$F1_{macro} = \frac{1}{C} \sum_{i=1}^{C} F1_i$$

Ejemplo:

Si volvemos al ejemplo anterior de los pacientes con cáncer, suponiendo que tenemos 100 muestras y 99 son negativas (no tienen cáncer):

- F1 de personas negativas: Acierta 99/99. Su F1 = 100%
- F1 de personas positivas: Como lo califico como que no tenía cáncer, sus verdaderos positivos son 0. Por lo tanto, F1 = 0%
- F1 Macro Total: La media entre los dos F1, quedaría que el F1 total es 50%

Esta es la utilidad del F1-Score Macro. Mientras el Accuracy nos habría dicho que el porcentaje de acierto es 100%, el F1-Score Macro nos dice la realidad: el porcentaje es 50% (solo acierta la mitad, en este caso, los que sí tienen cáncer)[18].

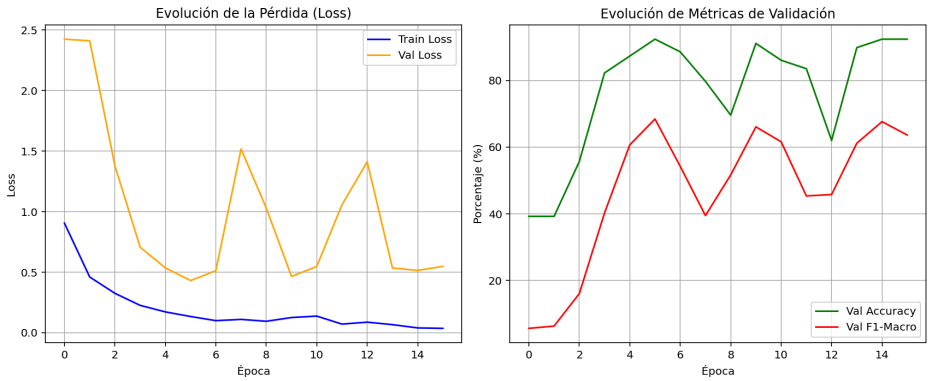
6.2. Experimento 1: Clasificación de Grano Grueso vs Clasificación de Grano Fino

Para poder analizar realmente la utilidad y la de las técnicas de Data Augmentation que hemos implementado, vamos a realizar dos clasificaciones distintas: uno de Grano Grueso (por tipos de objetos) y otro de Grano Fino (por modelos de objetos). Esto lo haremos con el modelo Baseline, es decir, con el DataSet original.

1. **Clasificación de grano grueso:** En esta ejecución, entrenamos al modelo para diferenciar entre objetos completamente diferentes entre ellos. Es decir, aprende a diferenciar bancos, papeleras, alcantarillas, etc. Como he explicado arriba este entrenamiento se ha hecho con el modelo PointNet utilizando solamente el conjunto de datos originales para entrenar el modelo.

```
--- Evaluación Final en Conjunto de TEST ---
RESULTADO FINAL TEST -> Loss: 0.3149 | Accuracy: 93.67% | F1-Macro: 76.52%
================================================================================

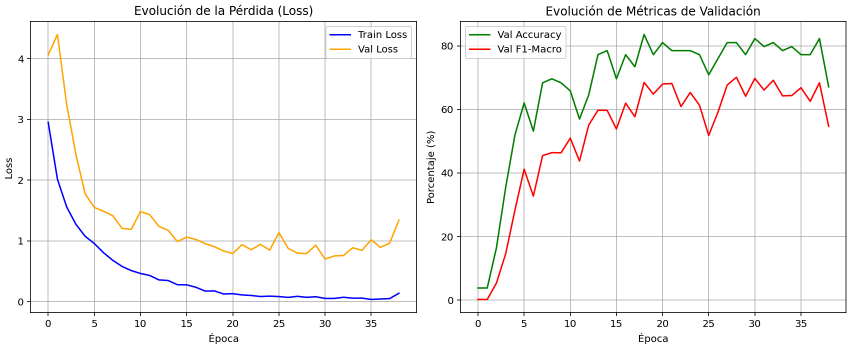
ANÁLISIS DE FALLOS EN TEST (5 errores):
ARCHIVO                                  | REALIDAD        | CONFUNDIDO CON
--------------------------------------------------------------------------------
23_9_2025_BinP1f_Segmented.npy           | Bin             | Bench
8_12_2025_Block3_Segmented.npy           | Block           | Bin
8_12_2025_Garbage4_Segmented.npy         | Garbage         | Bin
18_10_2025_SewerK_Segmented.npy          | Sewer           | Bench
7_12_2025_Structure1_Segmented.npy       | Structure       | Bin
================================================================================
```



1. **Clasificación de grano fino:** Esta clasificación es bastante más difícil que la anterior. Aqui se le exige al modelo, no solo saber diferenciar entre objetos diferentes, sino también identificar el modelo de esos objetos. Es decir, aprende a distinguir entre BancoB501, BancoB21, BancB53, etc. Al igual que en el anterior, también se ha utilizado el DataSet original.

```
--- Evaluación Final en Conjunto de TEST ---
RESULTADO FINAL TEST -> Loss: 0.9628 | Accuracy: 75.00% | F1-Macro: 61.14%
================================================================================

ANÁLISIS DE FALLOS EN TEST (23 errores):
ARCHIVO                                  | REALIDAD        | CONFUNDIDO CON
--------------------------------------------------------------------------------
1_11_2025_BenchB002c_segmented.npy       | BenchB002       | BenchB036
18_10_2025_BenchB006e_segmented.npy      | BenchB006       | BenchB008
19_10_2025_BenchB007n_segmented.npy      | BenchB007       | BenchB036
19_10_2025_BenchB030e_segmented.npy      | BenchB030       | BenchB036
28_9_2025_BenchB050b_Segmented.npy       | BenchB050       | BenchB030
28_9_2025_BenchB054b_Segmented.npy       | BenchB054       | BenchB007
28_9_2025_BenchB063a_segmented.npy       | BenchB063       | BenchB007
18_10_2025_BenchB079a_segmented.npy      | BenchB079       | BenchB825
18_10_2025_BenchB802c_segmented.npy      | BenchB802       | BenchB808
18_10_2025_BenchB825j_segmented.npy      | BenchB825       | BenchB832
18_10_2025_BenchB832a_segmented.npy      | BenchB832       | BenchB805
8_12_2025_BinP10c_Segmented.npy          | BinP10          | BinP2
19_10_2025_BinP12c_Segmented.npy         | BinP12          | BinP1
18_10_2025_BinP13j_Segmented.npy         | BinP13          | BinP4
9_10_2025_BinP26a_Segmented.npy          | BinP26          | BinP13
18_10_2025_BinP4d_Segmented.npy          | BinP4           | BinP2
7_12_2025_BinP6b_Segmented.npy           | BinP6           | MailBox
8_12_2025_Block4_Segmented.npy           | Block           | Stone
28_9_2025_Fountain2_Segmented.npy        | Fountain        | Pivot
6_12_2025_Fountain2_Segmented.npy        | Fountain        | Pivot
29_11_2025_Garbage4_segmented.npy        | Garbage         | MailBox
7_12_2025_MailBox1_Segmented.npy         | MailBox         | Garbage
7_12_2025_Structure2_Segmented.npy       | Structure       | Pivot
================================================================================
```



Cuando el objetivo consiste en identificar objetos totalmente diferentes entre sí (por ejemplo, distinguir entre “banco”, “papelera” o “alcantarilla”), la arquitectura PointNet demuestra que es capaz de hacerlo.

Sin embargo, vemos que la clasificación de grano grueso es más complicada y da un resultado bastante inferior al que daba en la separación por grano grueso. Al analizar los fallos nos damos cuenta de que la mayor cantidad de fallos son por confundirse en los modelos de bancos y papeleras, porque como ya hemos mencionado anteriormente, diferenciar entre modelos muy similares entre sí es mucho más complicado que diferenciar objetos que son totalmente distintos. Si nos fijamos en el gráfico, el F1-Macro no es capaz de superar el 65% en la parte de validación, y en el test final logra solo un 61,14%. Nuestro objetivo será utilizar Data Augmentation para mejorar estos resultados.

6.3. Experimento 2: PointNet vs PointNet++

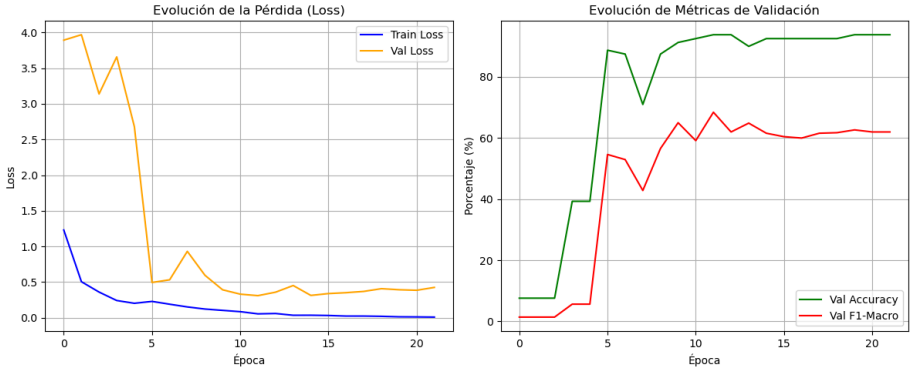
Tras analizar los resultados del Experimento 1, podemos ver que la arquitectura PointNet tiene claras dificultades para hacer correctamente la clasificación de grano fino. Esto se debe a que PointNet coge las características generales de la nube de puntos, pero no es capaz de detectar las pequeñas diferencias entre los objetos. Para solucionar este problema, en este segundo experimento introducimos la arquitectura PointNet++ y compararemos este resultado con el que nos ha dado el PointNet Vanilla.

En este experimento haremos lo mismo que hemos hecho en el Experimento 1 (utilizando el DataSet original sin Data Augmentation) pero utilizando el modelo PointNet++ y comparándolo con el Vanilla, tanto en grano grueso como en grano fino. Lo que hace PointNet++ es dividir el objeto en zonas más pequeñas y aplicar PointNet de forma recursiva. De esta forma, va juntando la información poco a poco y logra detectar detalles mucho más finos en la forma de los objetos [19].

1. **Clasificación de grano grueso:** Partiendo de la base de que PointNet ya conseguía un muy buen resultado, el objetivo aquí es comprobar que PointNet++ mantiene o supera ligeramente estos resultados. Como en esta clasificación la diferencia entre los diferentes objetos es muy grande, ambas arquitecturas consiguen un gran resultado y muy a la par.

```
--- Evaluación Final en Conjunto de TEST ---
RESULTADO FINAL TEST -> Loss: 0.1271 | Accuracy: 97.47% | F1-Macro: 79.58%
================================================================================

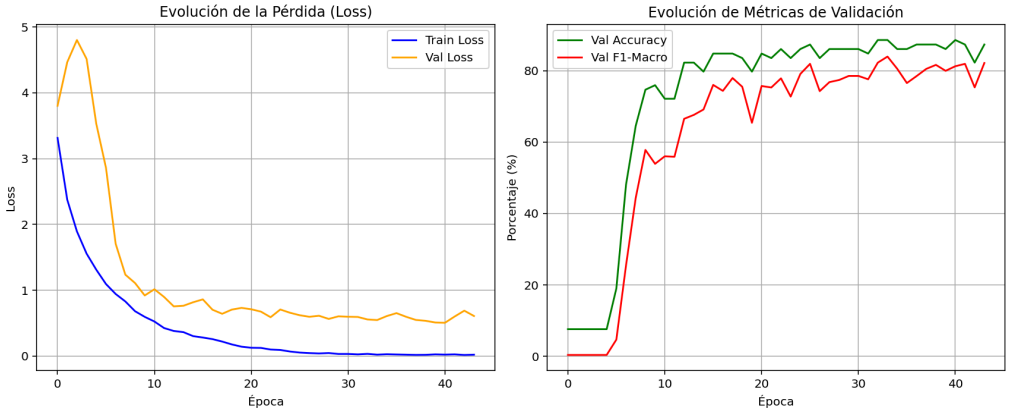
ANÁLISIS DE FALLOS EN TEST (2 errores):
ARCHIVO                                  | REALIDAD        | CONFUNDIDO CON
--------------------------------------------------------------------------------
8_12_2025_Block3_Segmented.npy           | Block           | Bin
7_12_2025_Structure1_Segmented.npy       | Structure       | Bin
================================================================================
```



1. **Clasificación de grano fino:** En esta clasificación es donde esperamos encontrar una gran diferencia en cuanto a rendimiento respecto al PointNet Vanilla. Al pedirle al modelo que distinga entre objetos muy parecidos (como el BancoB501, el BancoB21 o el BancB53), la capacidad de PointNet++ para fijarse en los pequeños detalles es muy efectiva.

```
--- Evaluación Final en Conjunto de TEST ---
RESULTADO FINAL TEST -> Loss: 0.5690 | Accuracy: 86.96% | F1-Macro: 74.45%
================================================================================

ANÁLISIS DE FALLOS EN TEST (12 errores):
ARCHIVO                                  | REALIDAD        | CONFUNDIDO CON
--------------------------------------------------------------------------------
18_10_2025_BenchB006e_segmented.npy      | BenchB006       | BenchB008
18_10_2025_BenchB036d_Segmented.npy      | BenchB036       | BenchB019
28_9_2025_BenchB050b_Segmented.npy       | BenchB050       | BenchB051
28_9_2025_BenchB054b_Segmented.npy       | BenchB054       | BenchB033
18_10_2025_BenchB079a_segmented.npy      | BenchB079       | Sewer
18_10_2025_BenchB832a_segmented.npy      | BenchB832       | BenchB805
19_10_2025_BinP12c_Segmented.npy         | BinP12          | BinP1
9_10_2025_BinP26a_Segmented.npy          | BinP26          | BinP13
18_10_2025_BinP4d_Segmented.npy          | BinP4           | Stone
8_12_2025_Block4_Segmented.npy           | Block           | ElectricityBox
28_9_2025_Fountain2_Segmented.npy        | Fountain        | Pivot
7_12_2025_MailBox1_Segmented.npy         | MailBox         | ElectricityBox
================================================================================
```



Mientras que PointNet tendía a confundir los modelos de bancos y papeleras debido a su similitud global (obteniendo un F1-Macro en test del 61,14%), la capacidad de PointNet++ para analizar las pequeñas diferencias es clave para conseguir un modelo mucho más preciso, pasando a lograr un F1-Macro en test del 74,45%.

6.4. Experimento 3: Aplicación de Data Augmentation para balanceo de DataSet

En este experimento vamos a analizar la utilidad de balancear el DataSet utilizando técnicas de Data Augmentation para mejorar el rendimiento del modelo. Además, vamos a hacer un estudio para ver cuáles son los parámetros óptimos para cada una de las técnicas implementadas. Es importante destacar que, debido a las limitaciones de recursos, todas las pruebas se han realizado utilizando el modelo PointNet Vanilla.

1. **Jittering:** Ahora veamos el resultado después de aplicar Jittering para balancear el DataSet. El resultado de ejecutar Jittering con distintos parámetros es el siguiente:

		CLIP						
		0,000000002	0,002	0,01	0,02	0,1	0,2	1
SIGMA	0,000000001	59,75	67,72	68,39	59,22	65,69	72,69	66,17
	0,001	70,58	63,36	61,67	72,8	64,49	68,14	67,58
	0,005	69,63	75,15	72,57	65,1	64,45	67,23	62,08
	0,01	70,53	74	74,19	70,21	67,81	60,56	67,77
	0,05	69,4	69,08	68,2	63,64	53,69	63,69	64,1
	0,1	64,75	72,99	73,49	72,57	72,2	54,24	60,26
	0,5	63,78	67,64	65,38	63,77	63,09	58,46	54,85

Como podemos ver en esta imagen, el punto óptimo está entre 0,005<sigma<0,1 y 0,002<clip<0,01. En este rango es capaz de mejorar significativamente el resultado base (baseline).

Sin embargo, si nos fijamos en la parte inferior izquierda, el resultado es muy malo, llegando incluso a empeorar en algunos casos los resultados del modelo original. Esto sucede porque cuando clip y sigma son demasiado grandes, las imágenes que se generar son demasiado diferentes al original, lo que puede confundir al modelo.

Por otro lado, si miramos los resultados de la primera fila o columna, el resultado es muy variado. Esto es porque los nuevos datos que se crean son muy parecidos a los datos originales, por lo que el resultado apenas varía respecto al Baseline. Es decir, no hay una tendencia clara de mejora o pérdida de precisión, como sí sucede en los dos casos anteriores.

2. **Rotation:** En este caso hemos probado a girar el objeto en varios ejes y con distinto rango de radios. El rango de radios representa los grados máximos que el objeto puede rotar:

		RANGO DE GRADOS			
		15º	45º	90º	180º
EJE DE ROTACION	Z	73,8	78,26	74,4	75,49
	Z, Y	69,01	67,07	60,63	64,4
	Z, Y, X	70,39	65,57	66,74	68,78

Esta imagen deja clara que uso óptimo de esta técnica consiste en rotar el objeto únicamente en el eje Z. La explicación de que esto suceda así es muy sencilla: en el contexto del mobiliario urbano todos los objetos están de pie, y por lo tanto lo único que puede variar (dependiendo de la perspectiva desde donde mires el objeto) es el eje Z. No tiene sentido entrenar al modelo con un banco dado la vuelta por completo porque nunca se va a encontrar con esa situación en el mundo real.

3. **Dropout:** Para esta técnica hemos entrado el modelo eliminando cantidades pequeñas de puntos y quitando una gran parte de los puntos, el resultado ha sido el siguiente:

PORCENTAJE DE NUMEROS ELIMINADOS				
10%	25%	50%	75%	95%
70,67	70,24	67,71	66,57	65,50

Aqui las conclusiones son claras: si eliminamos <25% de los puntos, el resultado mejora. Esto es porque logramos entrenar al modelo para que reconozca imágenes que no tienen tantos puntos como deberían, por ejemplo, por errores en la captura de datos. Sin embargo, cuando quitamos >75% el resultado empieza a ser bastante malo, porque al quitar tantos puntos el objeto se convierte en algo casi irreconocible, lo que puede confundir al modelo en el entrenamiento.

4. **Scale:** Probamos por último a hacer más grande o pequeño un objeto:

VARIACION			
5% (V=0.05)	10% (V=0.1)	25% (V=0.25)	50% (V=0.5)
66,19	64,96	66,42	67,66

Vemos que con esta técnica el resultado apenas varia, sino que se mantiene en el margen de 66% - 68%. El resultado mejora ligeramente respecto al modelo Baseline, gracias a que el DataSet se rebalancea, pero la técnica no demuestra ser especialmente útil

6.5. Experimento 4: Aplicación de Data Augmentation para aumento de DataSet

Por último, probaremos cada una de estas técnicas para multiplicar el Dataset y lo compararemos con el modelo Baseline:

		MULTIPLICADOR		
		x1	x2	x3
TECNICA	Original (sin DA)	61,14		
	Jittering	72,46	71,84	67,88
	Rotate	75,49	68,67	71,48
	Scale	70,46	69,32	70,56
	Dropout	66,3	68,73	69,1

Vemos que el resultado de multiplicar el DataSet no ha sido efectivo. Sin embargo, con el multiplicador x1 si hemos logrado mejorar significativamente el resultado del Baseline.

Esto puede ser debido al funcionamiento de la función Max Pooling que hemos explicado anteriormente: PointNet aprende a resumir el objeto sacando solo las características fundamentales (puntos críticos). Estos puntos críticos suelen ser aquellos que definen el contorno, los bordes afilados y las estructuras extremas del mobiliario urbano (el respaldo de un banco, el borde superior de una papelera). Los puntos que están en el centro del objeto no suelen ser tan relevantes y, por tanto, sus características son literalmente descartadas durante la función Max Pooling.

Esta función es la que logra que la red PointNet sea tan robusta frente a pequeñas variaciones, que es lo que se trata de generar al aplicar Data Augmentation. Por eso, el Data Augmentation no es tan efectivo utilizando este modelo. Sin embargo, cuando el objetivo es balancear el DataSet las técnicas de Data Augmentation son muy útiles para mejorar la precisión del modelo, porque evita que la red aprenda a decir siempre lo mismo (la clase mayoritaria).

7. Conclusiones

En este proyecto se ha demostrado la utilidad y la necesidad del Data Augmentation para mejorar el rendimiento de modelos de clasificación 3D en contextos de mobiliario urbano. A partir de los experimentos realizados, podemos extraer las siguientes conclusiones:

- **Solución al desbalanceo del DataSet:** La captura de un dataset propio en Vitoria-Gasteiz utilizando sensores LiDAR reflejó un problema que hay a la hora de conseguir datos para el entrenamiento de un modelo: la "distribución de cola larga". Se ha demostrado que utilizar técnicas de Data Augmentation para igualar la cantidad de datos de las clases minoritarias con las mayoritarias, mejora significativamente el rendimiento del modelo en la clasificación de grano fino.
- **Eficacia limitada de la multiplicación masiva:** Aunque el Data Augmentation ha demostrado ser muy útil a la hora de balancear el conjunto de datos, su uso para multiplicar el DataSet y aumentar su tamaño no ha resultado ser efectivo. La razón principal es que los datos utilizados para el test no presentan defectos ni ruido. Las técnicas de aumento de datos están diseñadas para simular alteraciones e imperfecciones propias de la captura de datos en un entorno real (por ejemplo, la visión de un coche autónomo, donde la velocidad del coche puede hacer que la captura no sea perfecta y que los puntos estén ligeramente movidos, lo cual se simula mediante Jittering). En nuestro caso, los datos capturados manualmente, tanto la parte de entrenamiento como la parte de test, eran impecables y no tenían ruido, por lo que entrenar a la red para que sea capaz de detectar defectos en la captura de datos, carecía de sentido y no se logró mejorar el resultado.
- **Aplicación de las técnicas según el contexto:** No todas las son útiles para entornos urbanos reales. Se ha demostrado, por ejemplo, que la técnica de Rotación solo mejora el rendimiento del modelo si se aplica sobre el eje Z. Esto sucede porque no tiene sentido entrenar un modelo con mobiliario urbano invertido o dado la vuelta.
- **Punto óptimo de generación de ruido:** Como se ha demostrado en el análisis de los parámetros en el experimento 3, es importante que la generación de ruido se haga de una manera controlada. Si los valores de esos parámetros son demasiado altos, corremos el riesgo de que el objeto sea completamente deformado, lo que puede confundir a la red y empeorar el rendimiento. La técnica de Jittering requiere aplicar parámetros muy bajos ($0.005 < sigma < 0.1$ y $0.002 < clip < 0.01$) para ser efectiva sin destruir la geometría general del objeto. De la misma forma, el Dropout resulta muy útil para simular oclusiones u obstáculos físicos si se elimina menos del 25% de los puntos, pero confunde por completo a la red si la pérdida supera el 75%.

En definitiva, las técnicas de Data Augmentation no deben entenderse como un mero multiplicador, sino como una herramienta útil para solucionar problemas concretos del dataset. Su uso para el balanceo evita que arquitecturas robustas como PointNet memoricen solamente las clases mayoritarias, lo cual la convierte en una herramienta

muy útil para lograr un modelo que clasifique también correctamente los objetos poco frecuentes.

8. Bibliografía

1. Congreso Estudiantil de Inteligencia Artificial Aplicada a la Ingeniería y Tecnología, UNAM, FESC, Estado de México, 2024. “Deteccion de objetos 3D con PointNet para la conducción autónoma”.

2. Llamas Valenzuela, Alma Carmina. "Predicción de Expansión Urbana y Detección de Cambio de Uso de Suelo Mediante Segmentación de Imágenes Satelitales y Técnicas de Machine Learning". Universidad Autónoma de Zacatecas

3. Moorthy C.V.K.N.S.N. Venkateswarulu N. et al Padmaja, B. Exploration of issues, challenges and latest developments in autonomous cars. Springer Open, 2023.

4. "Algoritmo para la clasificación de nubes de puntos LiDAR en entornos urbanos: discriminación entre vegetación y edificaciones.". I Congreso Iberoamericano de Geomática y Ciencias de la Tierra. X Topcart 2012.

5. Universidad Pedagógica y Tecnológica de Colombia, "Estado de arte y aplicación de técnicas de Aprendizaje Profundo desde imágenes reconstruidas a partir de nubes de puntos de sensores de Radar y Lidar"

6. Rodríguez Espina, Adrián. "Data Augmentation para Problemas de Object Detection”. Trabajo Fin de Grado, E.T.S.I. de Sistemas Informáticos (UPM)

7. Morer, P., & Rodríguez, M. (2017). Geometría computacional aplicada a la transformación y rotación de modelos tridimensionales.

8. Alejandro Areta Diaz, 2016. "Análisis y reconstrucción 3D de entornos complejos mediante múltiples sensores". Trabajo de Fin de Grado, UNIVERSIDAD CARLOS III DE MADRID. https://hdl.handle.net/10016/26888

9. Mundo Earth, "CloudCompare TUTOR 3. Segmentación y recorte de nubes de puntos", YouTube, 11 de junio de 2021. Disponible en: https://youtu.be/_yart5SmBCQ?si=m2sNkQz36S69RLAm

10. Rahul Vigneswaran, Marc T. Law, Vineeth N. Balasubramanian, Makarand Tapaswi. "Feature generation for long-tail classification". Proceedings of the Twelfth Indian Conference on Computer Vision, Graphics and Image Processing

11. Tecnológico Nacional de México. "Dataset de clasificación de Residuos Sólidos Urbanos para Redes Neuronales Convolucionales".

12. yanx27, repositorio de Github "Pointnet_Pointnet2_pytorch". Disponible en: https://github.com/yanx27/Pointnet_Pointnet2_pytorch.git

13. 2016, International Joint Conference on Neural Networks (IJCNN). "PointNet: A 3D Convolutional Neural Network for real-time object class recognition".

14. charlesq34, Repositorio de Github “PointNet”. Disponible en: https://github.com/charlesq34/pointnet/blob/master/provider.py

15. I. Goodfellow, Y. Bengio, y A. Courville, Deep Learning. MIT Press, 2016.

16. Lutz Prechelt. "Automatic early stopping using cross validation: quantifying the criteria". Fakultät für Informatik, Universität Karlsruhe, Karlsruhe, Germany.

17. Campo Yepes, John Jairo, 2026. "Evaluación de métricas en modelos predictivos de clasificación en machine learning". https://hdl.handle.net/10882/19105

18. Liliana Puente-Maury, Asdrúbal López-Chau, William Cruz-Santos, Lourdes López-García. "Método rápido de preprocesamiento para clasificación en conjuntos de datos no balanceados". Universidad Autónoma Metropolitana, Unidad Cuajimalpa, Distrito Federal, México

19. Alejandro Morales-Martín, Francisco Javier Mesas Carrascosa, Fernando Pérez Porras, Víctor Manuel Vargas, Francisco Moreno-Cano, Pedro Antonio Gutiérrez. "Evaluación del potencial de los modelos de Deep Learning para la clasificación de árboles a nivel de género utilizando datos LiDAR". Universidad de Córdoba, 2026. https://doi.org/10.17398/3101-7177.2.135